

# Examining Optimal Cislunar Invariant Manifolds in the Circular Restricted Three-Body Problem



Maximilian A. Slavik

Spaceflight Mechanics Project

Professor Sandeep Singh

Rensselaer Polytechnic Institute

April 2026

# 1 Introduction and Historical Background

As the exploration of Lunar space becomes more prevalent in the twenty first century, the examination of Earth-Moon libration point orbits is ever important. These libration point orbits allow for the design of specialized missions that make surveying, communication, crew operations, and satellite constellations possible around our nearest neighbor. This report considers the trajectories emanating from periodic orbits in the Circular Restricted 3 Body Problem (CR3BP). When periodic orbits are considered hyperbolic fixed points within the CR3BP phase space, stable and unstable trajectories can be formed by propagating disturbances along the respective eigenvectors of the Monodromy matrix. The set containing these trajectories is known as an invariant manifold. These manifolds function as pathways within the CR3BP such that a spacecraft can easily navigate between multiple periodic orbits [2]. To transfer between multiple periodic orbits, it is also possible to utilize both orbits' manifolds by performing a rendezvous maneuver between the two at an optimal impulse point. Invariant manifold "piercing points" with respect to the Earth and Moon  $x - z$  planes are identified and filtered, in order to locate prime candidates for Cislunar mission design.

## 2 The Circular Restricted Three Body Problem

The algorithm presented in this paper was developed according to Reference [1]. Consider a rotating reference frame centered upon the Barycenter between the Earth and Moon. The line joining the Earth and Moon is parallel to the X axis, with the Earth on the negative side at  $x_1 = -\mu$ , and the Moon at  $x_2 = (1 - \mu)$ , where  $\mu = 1.2150585 \cdot 10^{-2}$ . A dynamical system is derived in this rotating frame by a Hamiltonian formed from the gravitational potentials of the Earth and Moon, as well as the centrifugal and coriolis potentials inherent to the rotating frame. The results of Hamilton's equations are shown below:

$$\ddot{x} = x + 2\dot{y} - \frac{(1 - \mu)(x + \mu)}{r_1^3} - \frac{\mu(x - (1 - \mu))}{r_2^3}$$

$$\ddot{y} = y - 2\dot{x} - \frac{(1 - \mu)y}{r_1^3} - \frac{\mu y}{r_2^3}$$

$$\ddot{z} = -\frac{(1 - \mu)z}{r_1^3} - \frac{\mu z}{r_2^3}$$

Where  $r_1 = \sqrt{(x + \mu)^2 + y^2 + z^2}$  and  $r_2 = \sqrt{(x - 1 + \mu)^2 + y^2 + z^2}$  are the magnitudes of the distance vectors from the spacecraft to the Earth and Moon respectively.

From these equations, a single initial position and velocity vector determines a periodic orbit. The JPL Three Body Periodic Orbit Database [3] was utilized to gather precise initial conditions for a wide variety of cislunar orbit families, most notably of the Halo, Dragonfly, and Axial types. These orbits were propagated using Scipy's *SolveIVP* function over one orbital period to form phase space trajectory data for the entire orbit.

### 3 Methodology

Consider a known periodic orbit phase space trajectory  $\vec{S}[i] \in \mathbb{R}^6$ , where  $i$  is an index that corresponds to the evaluation point along the orbit. A time value is associated with  $i$  during numerical integration, which allows  $S$  to be expressed as a function of time. The set of state transition matrices  $\hat{\Phi}(0, t)$  that operate on the system state as  $\hat{\Phi}(0, t)\vec{S}(0) = \vec{S}(t)$  can be extrapolated from the position data and CR3BP Jacobian. This Jacobian is purely a function of position, and forms a dynamical system  $\frac{d\hat{\Phi}}{dt} = \hat{J}\hat{\Phi}$  with the state transition matrix. From this dynamical system,  $\hat{\Phi}$  can be evaluated along the orbit for one period  $T$ . The resultant state transition matrix evaluated at the exact periodic completion point is known as the Monodromy matrix  $\hat{M} = \hat{\Phi}(0, T)$ . Four out of six Eigenvectors of this matrix denote the direction sets in which stable, unstable, and central manifolds emanate from, locally to the relevant point in the periodic orbit. Out of the six Monodromy matrix Eigenvalues, the relevant two are extracted by eliminating complex Eigenvalues, then eliminating the Eigenvalue pair of  $(1, 1)$  within numerical tolerance. This step leaves only the reciprocal stable/unstable pair  $(\lambda_s, \lambda_u)$ , with  $\lambda_u > 1$  and  $\lambda_s < 1$ . Identifying Eigenvectors  $(\vec{v}_s, \vec{v}_u)$  that correspond to these reciprocal Eigenvalues subsequently associates the relevant Eigenvectors to either the stable or unstable manifold.

Utilizing the previously evaluated state transition matrices to propagate said Eigenvectors through an entire orbit, a set of perturbation conditions is found, corresponding to the original  $\vec{S}[i]$  periodic orbit states:

$$\vec{\delta}_s[i] = \frac{\vec{v}_s}{|\vec{v}_s|} \cdot 10^{-6} \ , \quad \vec{\delta}_u[i] = \frac{\vec{v}_u}{|\vec{v}_u|} \cdot 10^{-6}$$

For each  $i$ , a set of two initial conditions  $\vec{\eta}_m[i] = \vec{S}[i] + \vec{\delta}_m[i]$ , with  $m$  denoting either the stable or unstable vector. For each manifold, forward-time (+) and reverse-time (−) solutions are present, and both must be propagated to represent the full manifold behavior. During propagation of the stable and unstable manifolds, piercing points are identified for  $x = -\mu$  and  $x = 1 - \mu$  respectively, such that a mission-optimal region of phase space is located for spacecraft transfer orbit insertion.

## 4 Code Overview

Coding was performed in Python, using the Visual Studio Code IDE. A Runge-Kutta 45 solver was selected for propagation of dynamical systems within the algorithm, including the general CR3BP solver, state transition matrix propagation, and invariant manifold elements. Ten separate functions solve sub-sections of the main algorithm, with the final *InvariantManifolds* function calculating the final invariant manifold propagation data. To evaluate piercing points, Scipy *SolveIVP* was passed with dedicated event functions for piercing the Earth and Moon  $x-z$  planes. Using these event calls, *SolveIVP* identifies the time at which the event function's root was passed, then subsequently uses a root finder to precisely locate the root point in time. Four solution bunch objects are returned from *InvariantManifolds*, capturing the propagation for the  $+$  and  $-$  branches of the stable and unstable manifolds emanating from the initial periodic orbit.

Code for the CR3BP Propagator is shown below. Full algorithm code is located in the Appendix at the end of this report.

```
def CR3BP(t, S0vec):

    x, y, z = S0vec[0], S0vec[1], S0vec[2]
    xdot, ydot, zdot = S0vec[3], S0vec[4], S0vec[5]

    R1 = np.sqrt((x + mu)**2 + y**2 + z**2)
    R2 = np.sqrt((x - 1 + mu)**2 + y**2 + z**2)

    xddot = (x + 2*ydot - (1 - mu)*(x + mu)/(R1**3)
              - mu*(x - 1 + mu)/(R2**3))

    yddot = (y - 2*xdot - y*((1 - mu)/(R1**3) + (mu)/(R2**3)))

    zddot = -((1 - mu)/(R1**3) + (mu)/(R2**3))*z

    S0dot = np.array([xdot, ydot, zdot, xddot, yddot, zddot])

    return S0dot
```

## 5 Results

Invariant manifold pathways with piercing points were applied for a Dragonfly orbit and an L1-Axial orbit to demonstrate mission insertion orbits for both manifold types. Invariant manifolds without piercing points were propagated for Dragonfly and L2-Halo orbits to showcase the existence of zero-velocity boundaries and other sub-structures of the CR3BP.

Results are shown in plots of manifold trajectories below. To aid in providing detailed, interactive plot settings, the Claude Sonnet LLM was utilized.

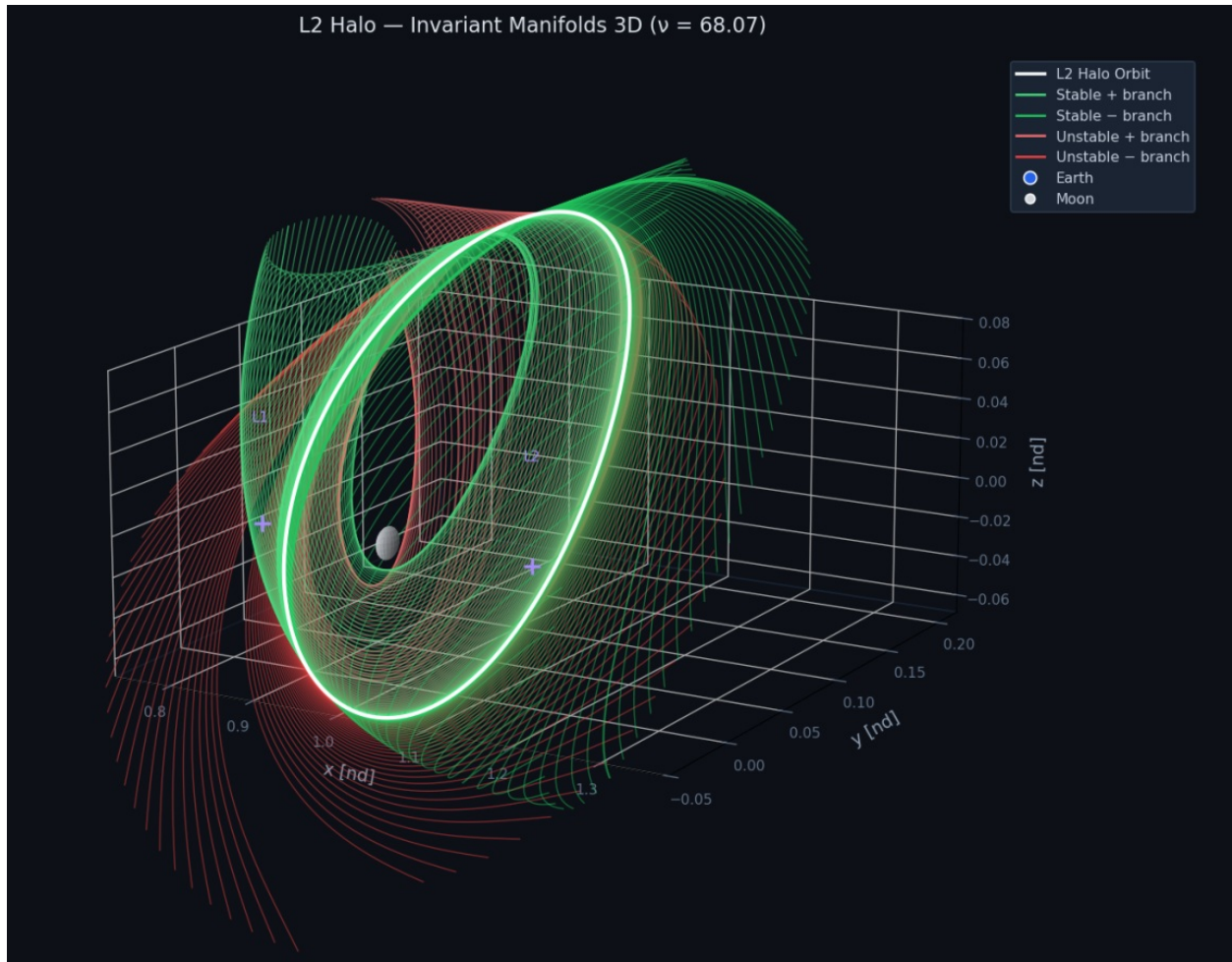


Figure 1: Arrival/Departure of Invariant Manifolds from an L2 Halo Orbit

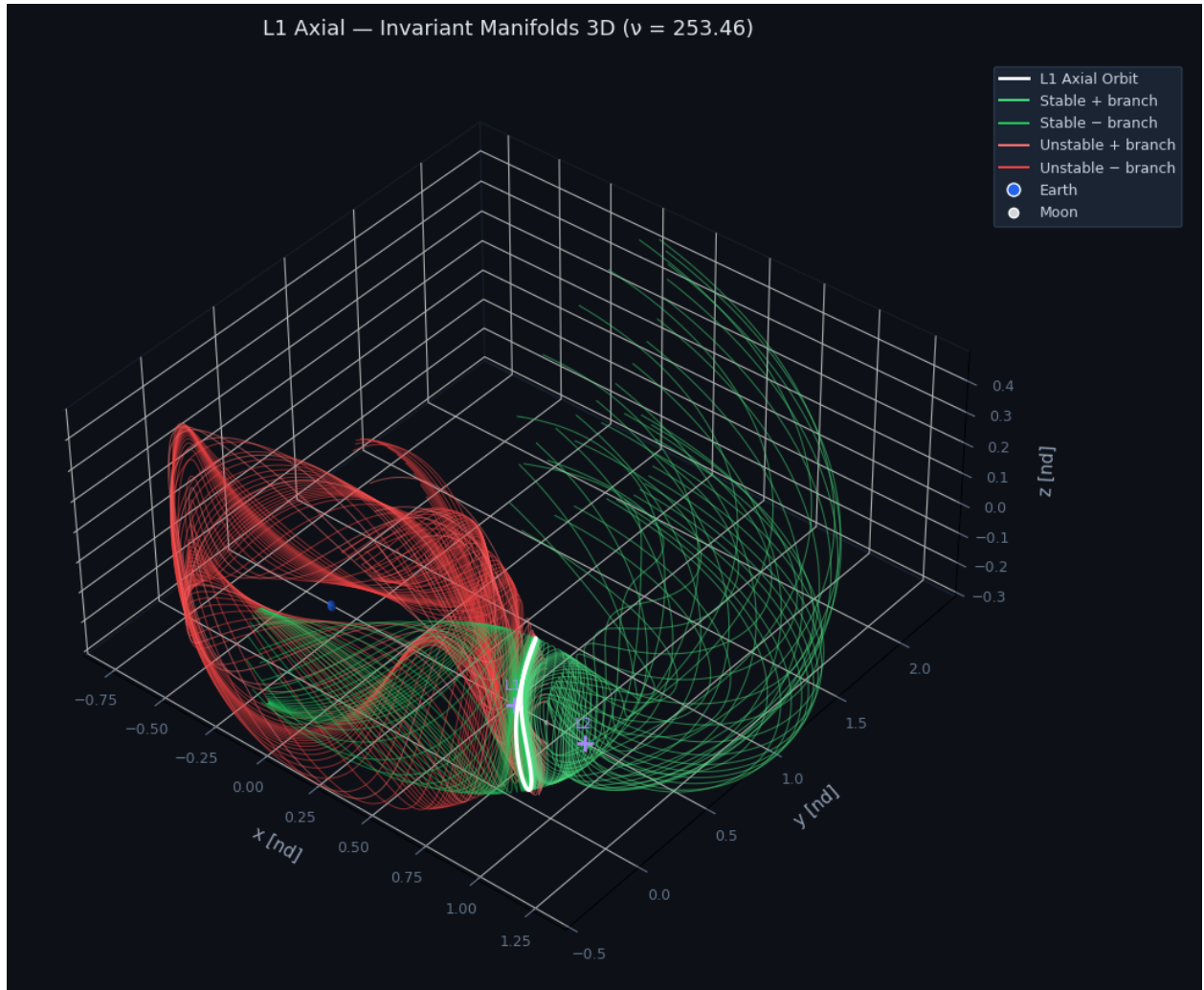


Figure 2: Manifold Propagation with Piercing Points for an L1 Axial Orbit (3D)

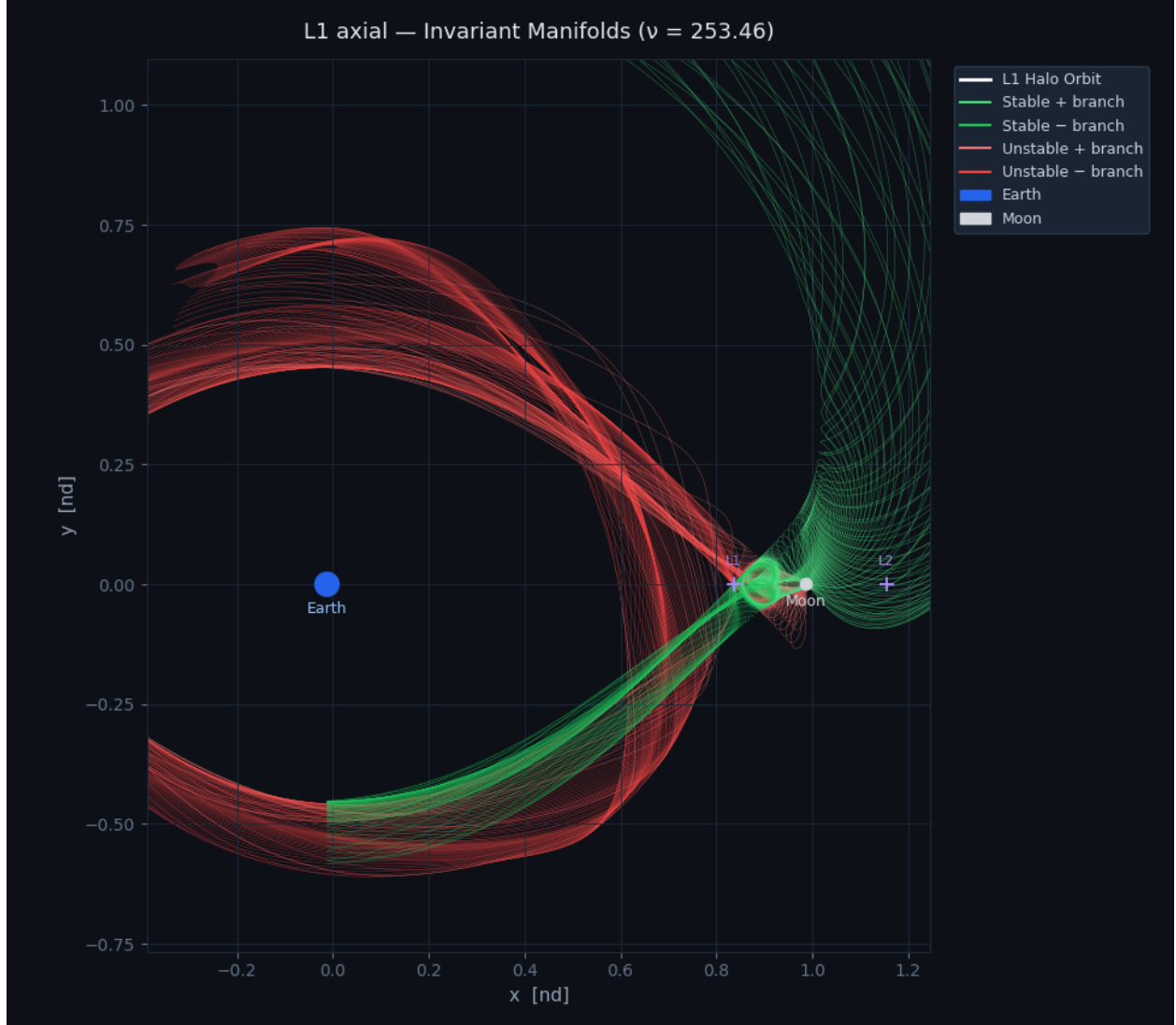


Figure 3: Manifold Propagation with Piercing Points for an L1 Axial Orbit (2D Projection)

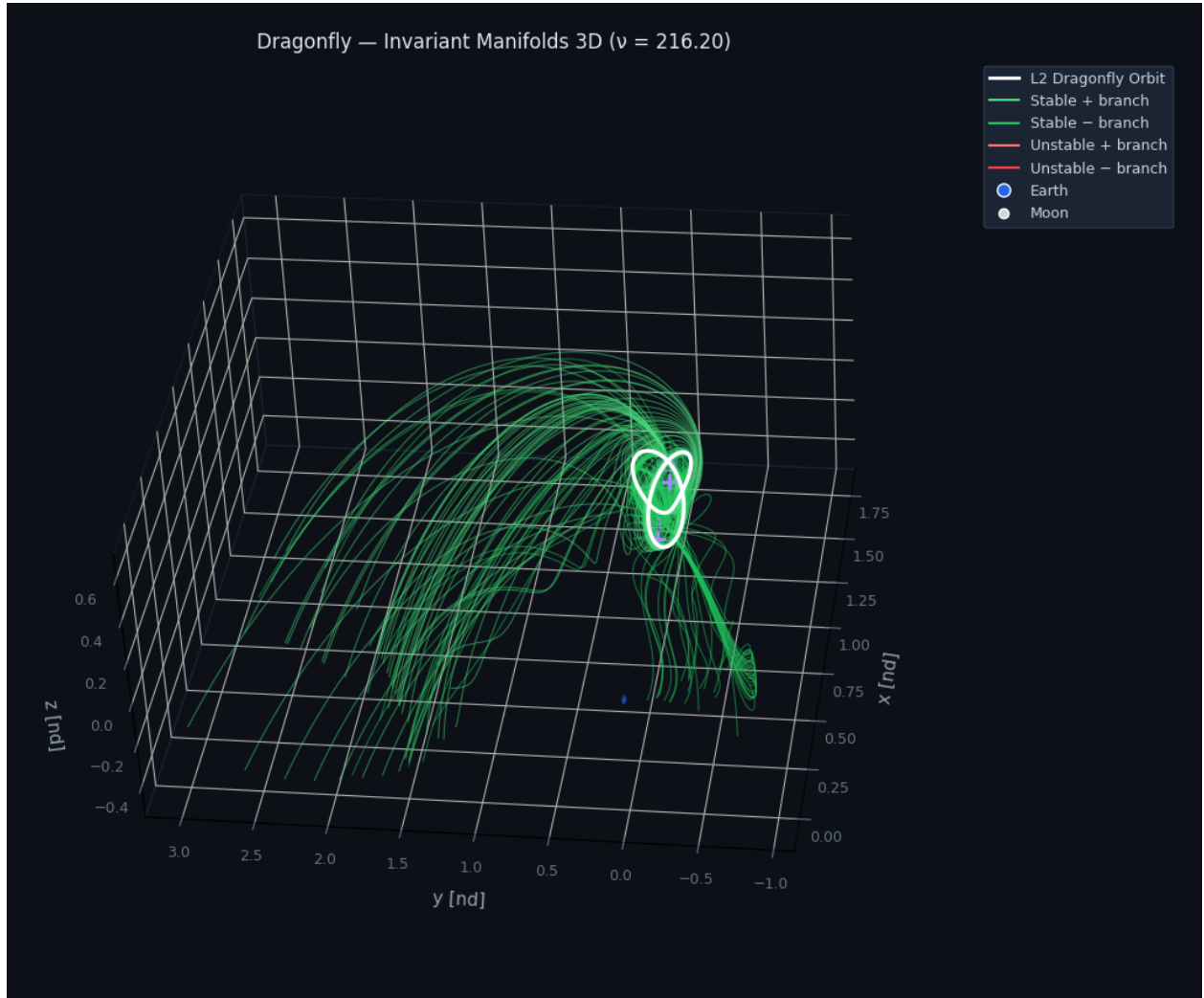


Figure 4: Manifold Propagation with Piercing Points for a Dragonfly Orbit (3D)

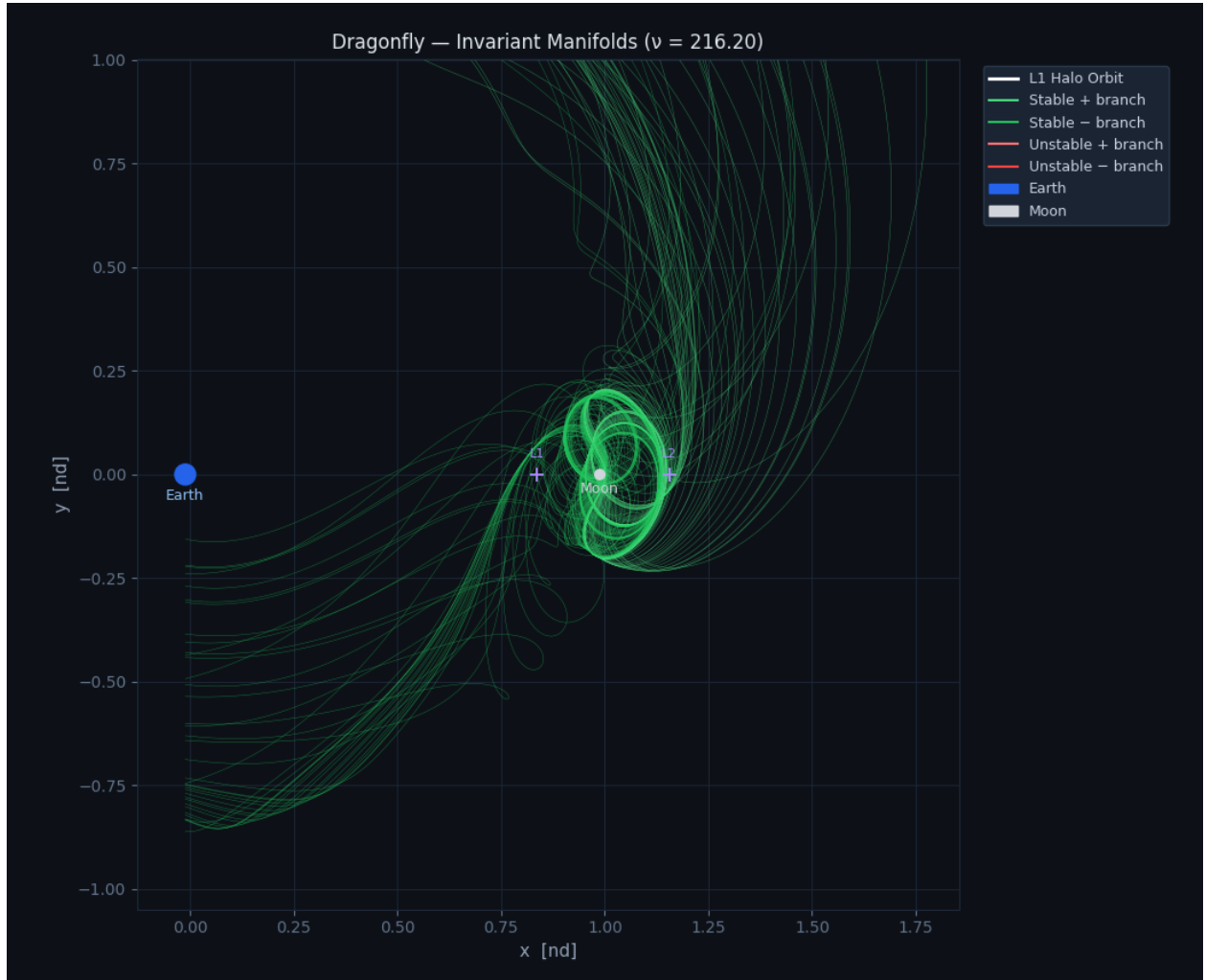


Figure 5: Manifold Propagation with Piercing Points for a Dragonfly Orbit (2D Projection)

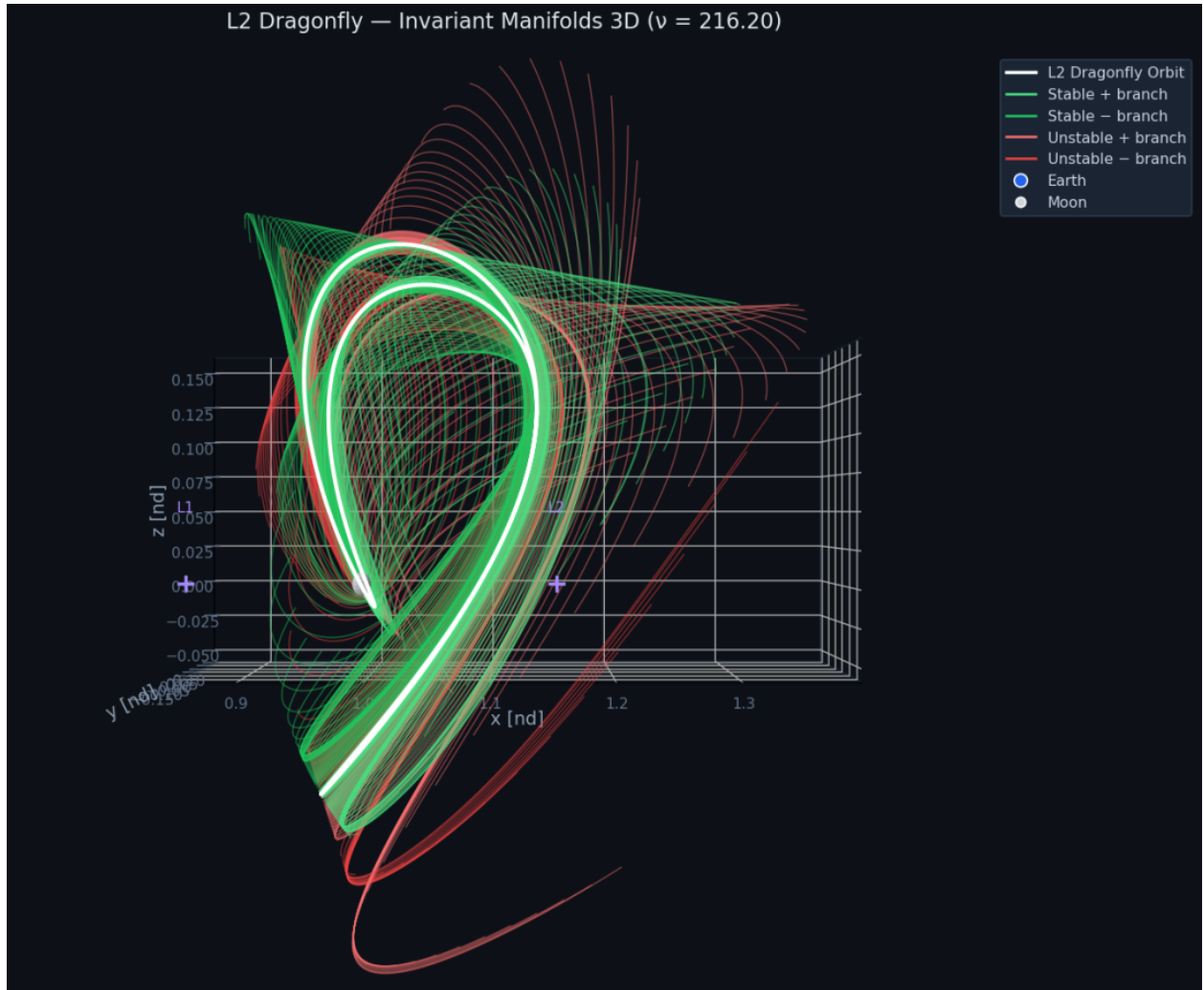


Figure 6: Arrival/Departure of Invariant Manifolds from a Dragonfly Orbit

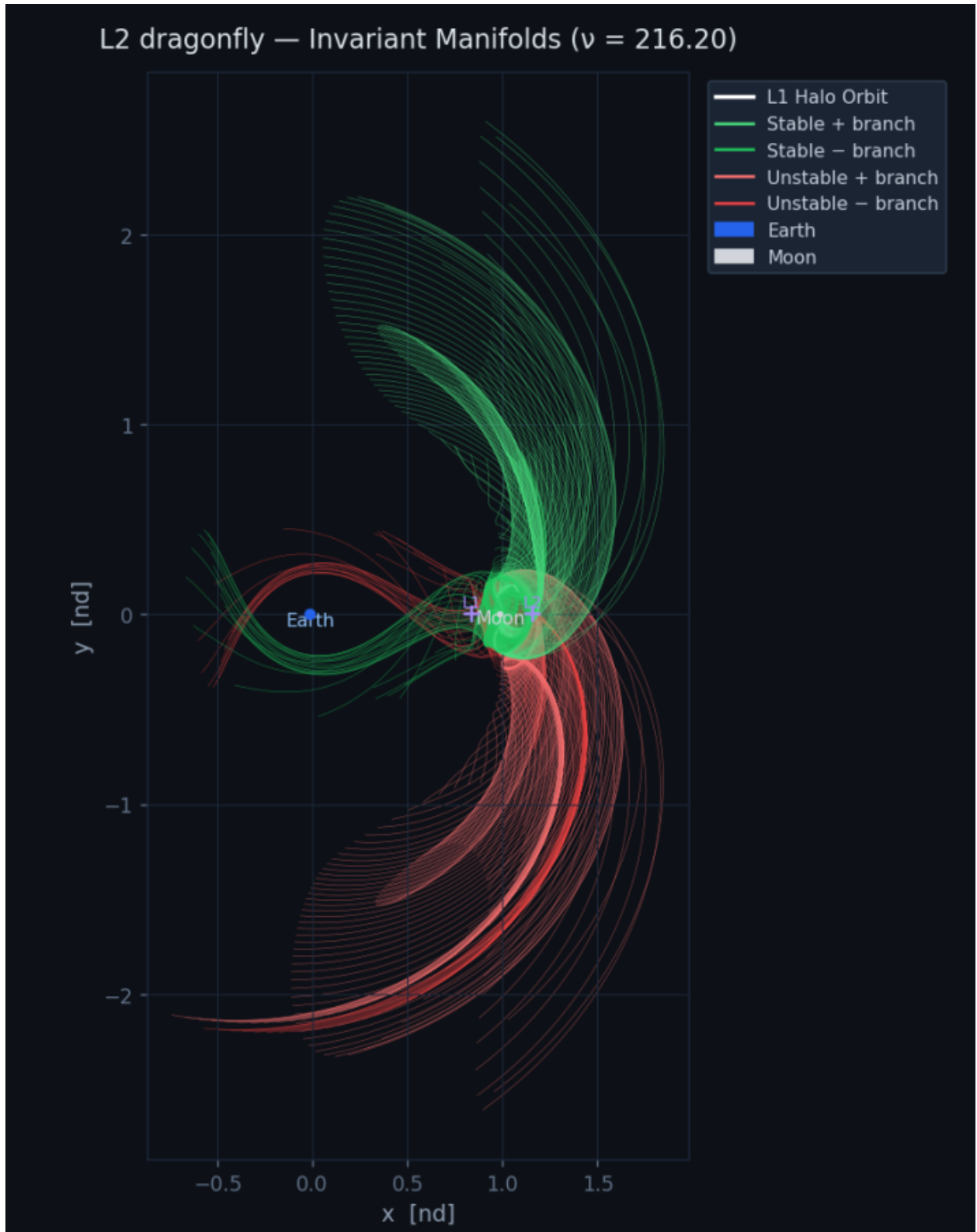


Figure 7: Invariant Manifolds for a Dragonfly Orbit, Showcasing a Zero-Velocity Boundary

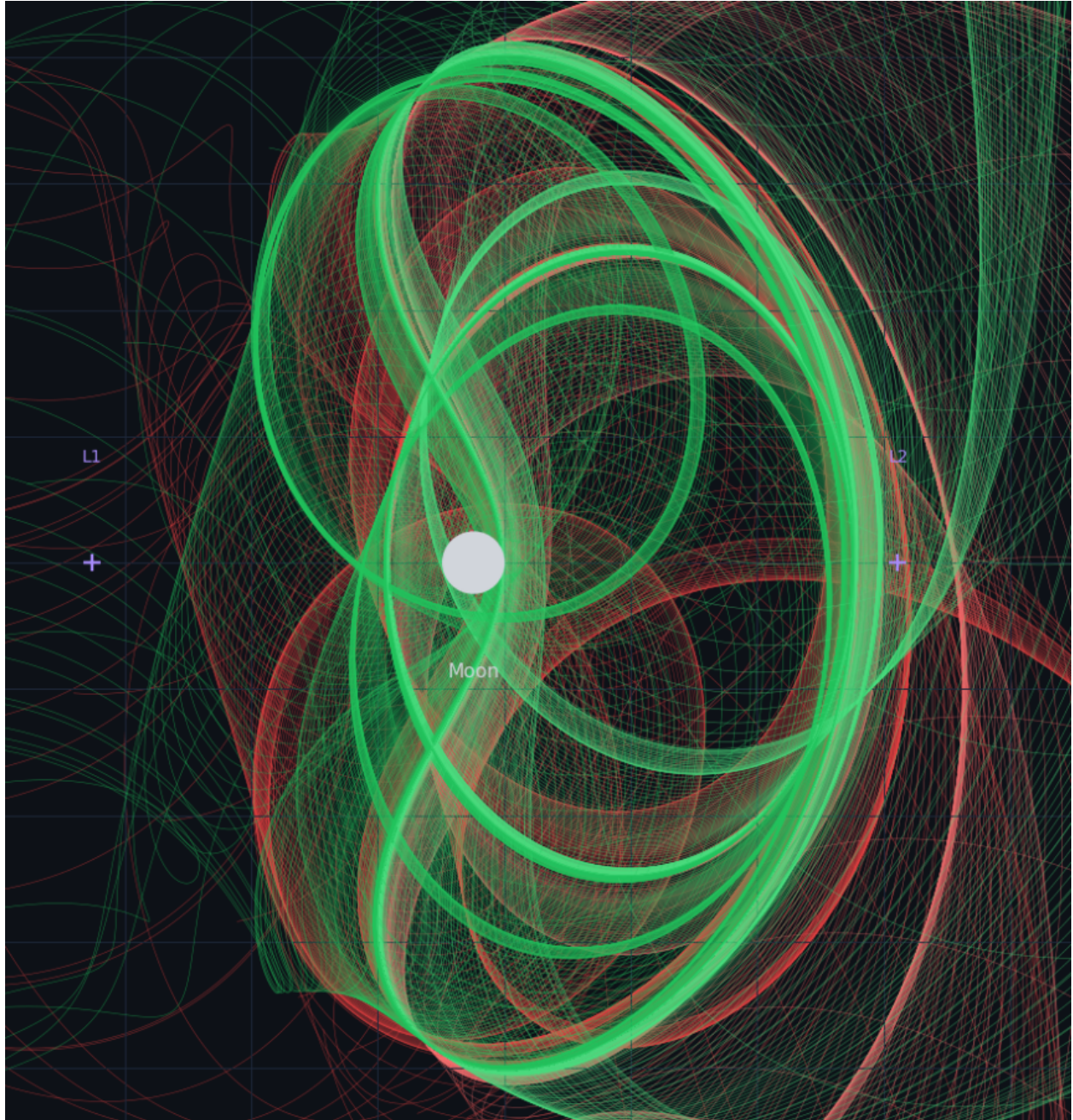


Figure 8: Local Behavior of Invariant Manifolds for a Dragonfly Orbit

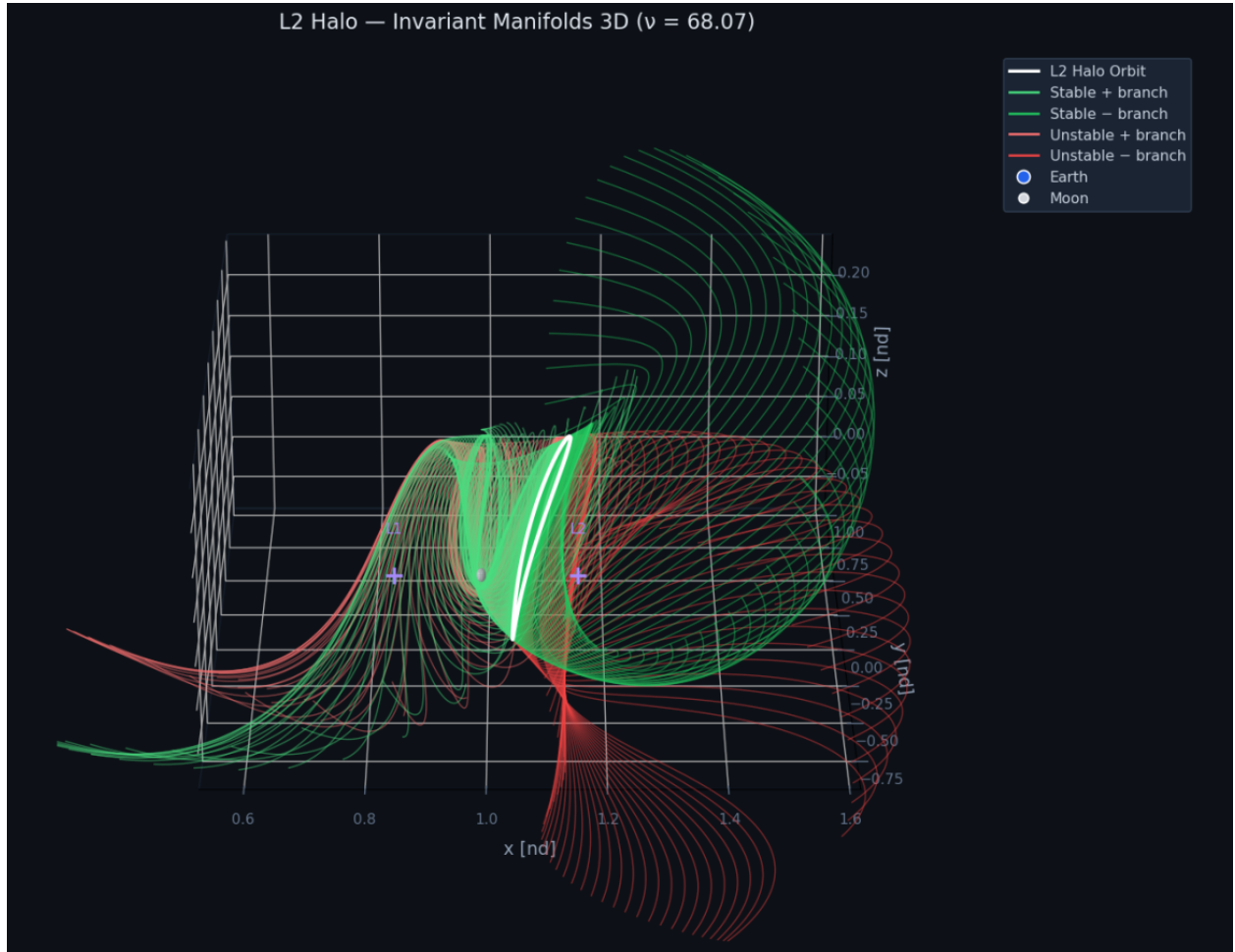


Figure 9: Arrival/Departure of Invariant Manifolds from an L2-Halo Orbit (Side View)

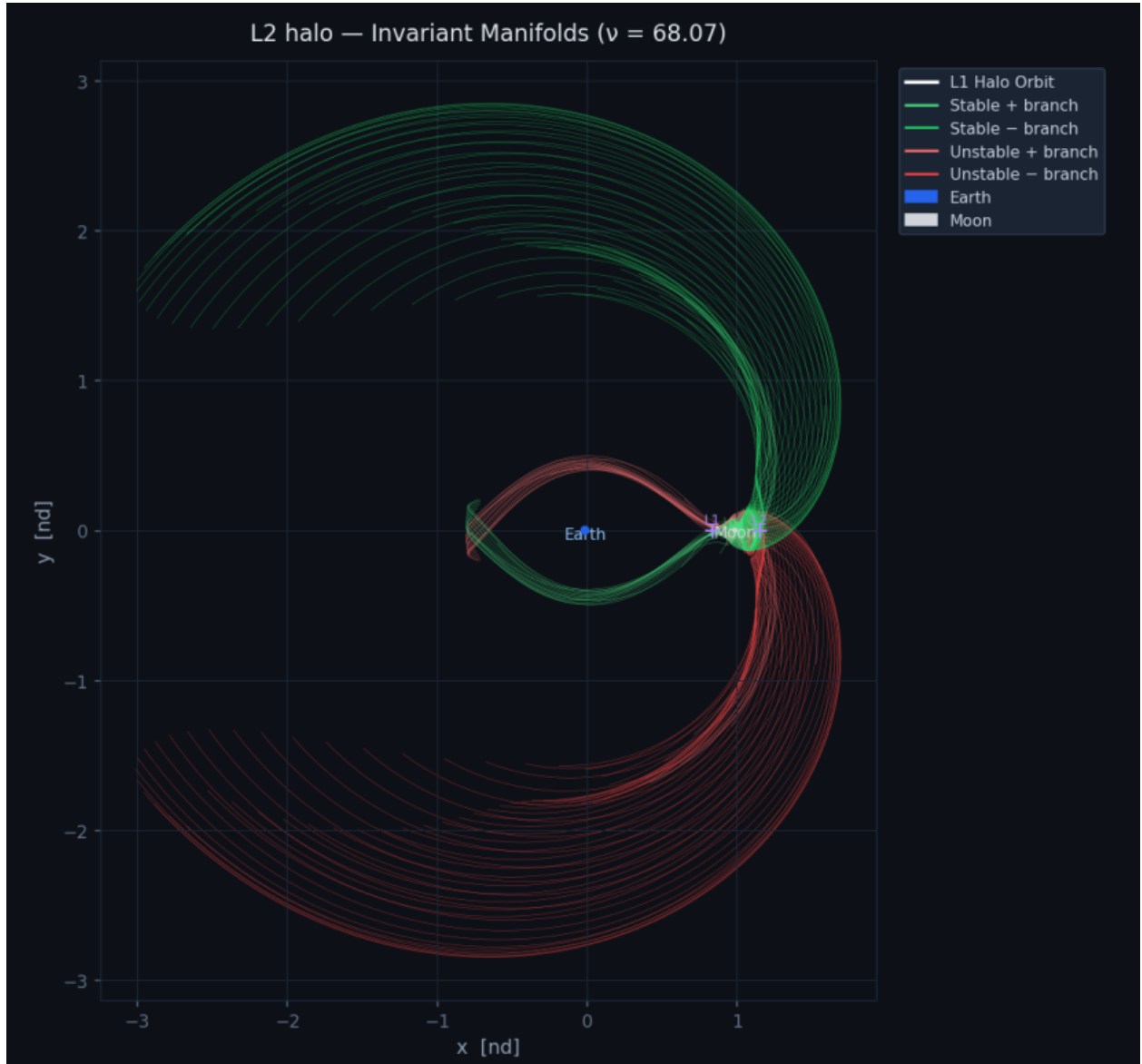


Figure 10: Invariant Manifolds for an L2 Halo Orbit, Showcasing a Zero-Velocity Boundary

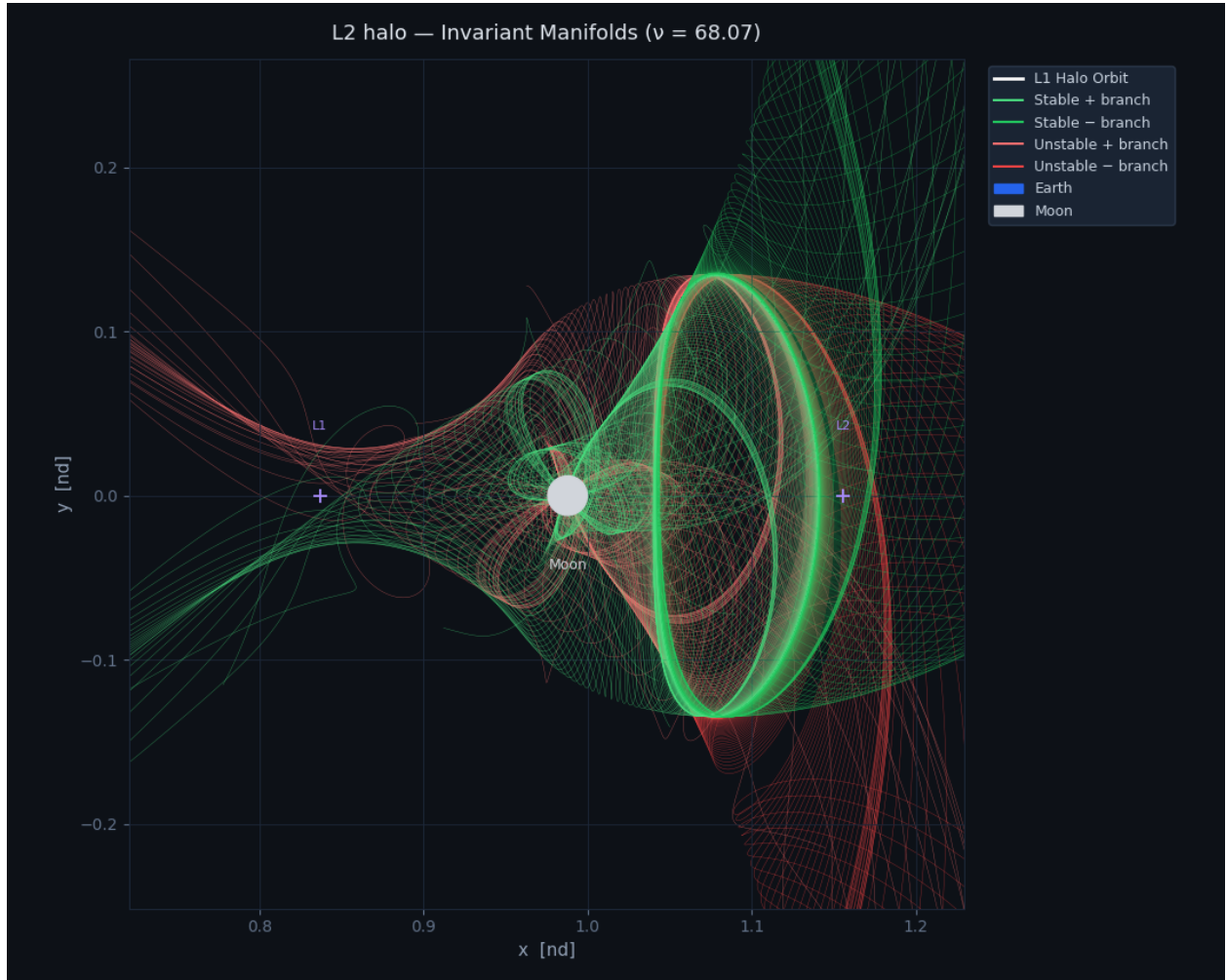


Figure 11: Local Behavior of Invariant Manifolds for an L2 Halo Orbit

## 6 Conclusion

Invariant manifolds of the Earth-Moon CR3BP offer efficient pathways for accessing a wide range of periodic cislunar orbits. When propagated, these manifolds also reveal underlying structures in the system, such as zero-velocity boundaries constrained by the Jacobi constant, and weak stability regions near Lunar space.

## 7 Lessons Learned

Originally this project aimed to identify invariant manifolds for multiple periodic orbits, filter inertial reference frame energies to reveal ideal pathways, and subsequently solve a Hamiltonian optimal control problem for the manifold transfer impulses based on the Pontryagin maximum principle. Due to significant time limitations, this goal was not able to be reached. In retrospect, a much simpler project should have been selected. However, a great deal was learned about the mathematical basis for invariant manifolds in dynamical systems, and the project was successful in its later re-framed goal of computing invariant manifolds with piercing points for various periodic orbits. Originally the indices used for propagation of the initial periodic orbit were used again for the manifold propagation initial conditions. This coupling made it highly impractical to rapidly test the algorithm, as a low number of points would result in a chaotic divergence from the periodic orbit, while a high number of points increased computation time by multiple orders of magnitude due to the sheer number of manifold elements that would need propagation. This issue was solved by decoupling the indices into two sets, an extremely fine  $n_f \approx 100,000$  for periodic orbit and state transition matrix propagation, and the coarse  $n_p \approx 50$  used in a *linspace* array built from  $n_f$ . The coarse array was subsequently used to propagate the invariant manifolds, decoupling the bulk of computation from the necessary precision earlier in the algorithm. Overall, this improvement saved a lot of time toward the latter half of project development.

## 8 Future Work

Future development of this project will focus on implementing an optimal control problem and two-point boundary value problem. To achieve this efficiently, the existing piercing points will be transformed to the inertial frame of the relevant body and filtered by orbital energy. The optimal control problem will then be implemented through Pontryagin's maximum principle, with the control law solved numerically using the Scipy *SolveBVP* function. The resulting control law will be propagated through the CR3BP to verify its ability to reach the target orbit.

## References

- [1] Kevin I. Alvarado and Sandeep K. Singh. Exploring the design space of low-thrust transfers with ballistic terminal coast segments in cis-lunar space. *Aerospace*, 12(3):217, 2025.
- [2] W. S. Koon, M. W. Lo, J. E. Marsden, S. D. Ross, G. Gómez, and J. Masdemont. Invariant manifolds, the spatial three-body problem and space mission design. *AAS/AIAA Astrodynamics Specialist Conference*, 2001.
- [3] NASA Jet Propulsion Laboratory. Three-body periodic orbits database. [https://ssd.jpl.nasa.gov/tools/periodic\\_orbits.html](https://ssd.jpl.nasa.gov/tools/periodic_orbits.html), 2020. *Accessed* : 2026 – 04 – 29.

## 9 Appendix

### 9.1 Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import brentq
from scipy.integrate import solve_bvp
from scipy.integrate import solve_ivp
import matplotlib.patches as mpatches

import JPL_Orbits

mu = 1.2150585e-2

npts = 1000000

npts_fine = 100000

npts_period = 100
```

```

eps_manifold = 1e-6

# S0 = np.array([ 1 - mu - 0.037, 0.19, 0, 0.42, 0.15, 0.0])

def CR3BP(t, S0vec):

    x,y,z = S0vec[0], S0vec[1], S0vec[2]
    xdot,ydot,zdot = S0vec[3], S0vec[4], S0vec[5]

    R1 = np.sqrt((x + mu)**2 + y**2 + z**2)
    R2 = np.sqrt((x - 1 + mu)**2 + y**2 + z**2)

    xddot = (x + 2*ydot - (1 - mu)*(x + mu)/(R1**3)
              - mu*(x - 1 + mu)/(R2**3))

    yddot = (y - 2*xdot - y*((1 - mu)/(R1**3) + (mu)/(R2**3)))

    zddot = -((1 - mu)/(R1**3) + (mu)/(R2**3))*z

    S0dot = np.array([xdot,ydot,zdot,xddot,yddot,zddot])

    return S0dot

def reach_earth(t, s):
    return s[0] + mu

reach_earth.terminal = True

```

```

def reach_moon(t, s):
    return s[0] - (1 - mu)
reach_moon.terminal = True

def propagate_3bp(S0, t0, tfin, npts, event=None):

    t_eval = np.linspace(t0,tfin,npts)

    #
    sol3bp = solve_ivp(CR3BP,np.array([t0,tfin]),S0,'RK45',t_eval=t_eval,
        events=event, rtol=1e-12, atol=1e-12)

    return sol3bp

def udbl_matrix(x, y, z):
    '''
    spatial second derivative matrix d2u/dxidxj

    returned matrix is the bottom left of the jacobian for the cr3bp

    '''
    R1 = np.sqrt((x + mu)**2 + y**2 + z**2)
    R2 = np.sqrt((x - 1 + mu)**2 + y**2 + z**2)
    c1, c2 = (1 - mu)/R1**3, mu/R2**3
    d1, d2 = (1 - mu)/R1**5, mu/R2**5

    x1, x2 = x + mu, x - 1 + mu

    Uxx = 1 - c1 - c2 + 3*(d1*x1**2 + d2*x2**2)
    Uyy = 1 - c1 - c2 + 3*(d1*y**2 + d2*y**2)

```

```

Uzz = -c1 - c2 + 3*(d1*z**2 + d2*z**2)
Uxy = 3*y*(d1*x1 + d2*x2)
Uxz = 3*z*(d1*x1 + d2*x2)
Uyz = 3*y*z*(d1 + d2)

return np.array([[Uxx, Uxy, Uxz],[Uxy, Uyy, Uyz],[Uxz, Uyz, Uzz]])

def Jacobian(x,y,z):
    Jac3bp = np.zeros([6,6])

    udbl = udbl_matrix(x,y,z)
    Jac3bp[0,3],Jac3bp[1,4], Jac3bp[2,5] = 1,1,1
    Jac3bp[3:6, 0:3] = udbl
    Jac3bp[3, 4] = 2
    Jac3bp[4, 3] = -2

    return Jac3bp

def PreMonodromy(t,S0Total):

    dS3bp = CR3BP(t,S0Total[0:6])

    STM = S0Total[6:].reshape(6,6)

    x,y,z = S0Total[0],S0Total[1],S0Total[2]

    Stm2 = (Jacobian(x,y,z))@STM

    Stm2 = Stm2.flatten()

```

```

dS0Totaldt = np.zeros(len(S0Total))

dS0Totaldt[0:6] = dS3bp

dS0Totaldt[6:] = Stm2

return dS0Totaldt

def propogateSTM(S0,t0,tfin,npts):

    S0_Total = np.zeros(42)

    S0_Total[:6] = S0

    S0_Total[6:] = np.eye(6).flatten()
    t_eval = np.linspace(t0,tfin,npts)

    solSTM =
        solve_ivp(Premonodromy,np.array([t0,tfin]),S0_Total,'RK45',t_eval,rtol=1e-12,
            atol=1e-12)

    return solSTM

def EigsMpropagate(S0,t0,npts_fine,period):

    solSTM = propogateSTM(S0,t0,t0 + period,npts_fine)
    STMs = solSTM.y[6:,:]
    STMs = STMs.reshape(6,6,npts_fine)
    STMs = solSTM.y[6:,:].reshape(6, 6, npts_fine)

```

```

M = solSTM.y[6:,-1]
M = M.reshape(6,6)
eigs = np.linalg.eig(M)
eigvecs = eigs.eigenvectors
eigvals = eigs.eigenvalues

order = np.argsort(np.abs(eigvals))

lamda_u = eigvals[order[-1]]

nu = 0.5*(np.abs(lamda_u.real) + 1.0/np.abs(lamda_u.real))
print(f'Stability index = {nu:.4f}')

v_s = eigvecs[:, order[0]].real
v_u = eigvecs[:, order[-1]].real

v_s /= np.linalg.norm(v_s)
v_u /= np.linalg.norm(v_u)

stable_eigvecpropagate = np.zeros((npts_fine,6))

unstable_eigvecpropagate = np.zeros((npts_fine,6))

for i in range(npts_fine):
    S_eigvecstor,U_eigvecstor = STMs[:, :, i]@v_s, STMs[:, :, i]@v_u
    stable_eigvecpropagate[i, :], unstable_eigvecpropagate[i, :] =
        S_eigvecstor, U_eigvecstor

return stable_eigvecpropagate, unstable_eigvecpropagate

```

```

def InvariantManifolds(S0, t0, npts_fine, npts_period, period, IM_dt,
    IM_npts):

    psol = propagate_3bp(S0, t0, t0 + period, npts_fine)
    X_orbit = psol.y[:6, :].T
    S_eigvecs, U_eigvecs = EigsMpropagate(S0, t0, npts_fine, period)

    indices = np.linspace(0, npts_fine - 1, npts_period, dtype=int)

    solSP = []
    solSM = []
    solUP = []
    solUM = []

    print(f"Propagating {npts_period} manifold branches...")

    for i, idx in enumerate(indices):

        S_seed = X_orbit[idx, :]
        vs = S_eigvecs[idx, :]
        vu = U_eigvecs[idx, :]

        vs /= np.linalg.norm(vs)
        vu /= np.linalg.norm(vu)

        solSP.append(propagate_3bp(S_seed + 1e-6*vs, t0, t0 - IM_dt, IM_npts,
            event=reach_earth).y)
        solSM.append(propagate_3bp(S_seed - 1e-6*vs, t0, t0 - IM_dt, IM_npts,
            event=reach_earth).y)
        solUP.append(propagate_3bp(S_seed + 1e-6*vu, t0, t0 + IM_dt, IM_npts,

```

```

        event=reach_moon).y)
    solUM.append(propagate_3bp(S_seed - 1e-6*vu, t0, t0 + IM_dt, IM_npts,
        event=reach_moon).y)

    return solSP, solSM, solUP, solUM

##### dragonfly orbit test case

family = 'dragonfly'
Lset = 2

S0_act, period_act, info_act = JPL_Orbits.fetch_orbit(family=family,
    branch='N', idx=250)

family = 'axial'
Lset = 1

S0_act, period_act, info_act =
    JPL_Orbits.fetch_orbit(family=family, libr=Lset, idx=378)

IM_dt = 4.55*period_act
IM_npts = 120000

npts_period = 80

print(f"\nComputing manifolds for {npts_period} points on orbit...")

```

```

print(f"  Propagating  $\pm$ {IM_dt:.3f} TU ( $\sim$ {IM_dt * info_act['tunit_s'] /
      86400:.1f} days)")

solSP, solSM, solUP, solUM = InvariantManifolds(
    S0_act, 0.0, npts_fine, npts_period, period_act, IM_dt, IM_npts
)

print("Manifolds computed. Plotting...")


from mpl_toolkits.mplot3d import Axes3D


lw3d = 0.8


azim = -50


elev = 20


### PLOTTING CODE


print("Building 3D static plot...")


fig = plt.figure(figsize=(12, 9), facecolor='#0d1117')
ax = fig.add_subplot(111, projection='3d', facecolor='#0d1117')

```

```

# --- periodic orbit ---
orbit_sol = propagate_3bp(S0_act, 0.0, period_act, 500)
ax.plot(orbit_sol.y[0], orbit_sol.y[1], orbit_sol.y[2],
        color='#ffffff', lw=2.5, zorder=10, label=f'L{Lset}
        {family.capitalize()} Orbit')

# --- Earth sphere at (-mu, 0, 0) ---
u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]
r_earth = 0.016
x_earth = -mu + r_earth*np.cos(u)*np.sin(v)
y_earth =      r_earth*np.sin(u)*np.sin(v)
z_earth =      r_earth*np.cos(v)
ax.plot_surface(x_earth, y_earth, z_earth, color='#2563eb', alpha=0.9,
               zorder=15)

# --- Moon sphere at (1-mu, 0, 0) ---
r_moon = 0.008
x_moon = 1-mu + r_moon*np.cos(u)*np.sin(v)
y_moon =      r_moon*np.sin(u)*np.sin(v)
z_moon =      r_moon*np.cos(v)
ax.plot_surface(x_moon, y_moon, z_moon, color='#d1d5db', alpha=0.9, zorder=15)

# --- Lagrange points ---
L_pts = {'L1': (0.83692, 0, 0), 'L2': (1.15568, 0, 0)}
for lname, (lx, ly, lz) in L_pts.items():
    ax.scatter([lx], [ly], [lz], color='#a78bfa', marker='+', s=100,
               linewidths=2, zorder=12)
    ax.text(lx, ly, lz + 0.05, lname, color='#a78bfa', fontsize=9,
           ha='center')

# --- manifolds - sample for performance ---
step = max(1, npts_period // 40) # ~40 curves per branch

```

```

for i in range(0, npts_period, step):
    # stable manifolds
    # MODIFIED: List indexing required since arrays are now heterogeneous
    lengths
    ax.plot(solSP[i][0, :], solSP[i][1, :], solSP[i][2, :],
            color='#4ade80', lw=lw3d, alpha=0.5, zorder=2)
    ax.plot(solSM[i][0, :], solSM[i][1, :], solSM[i][2, :],
            color='#22c55e', lw=lw3d, alpha=0.5, zorder=2)

    # unstable manifolds
    ax.plot(solUP[i][0, :], solUP[i][1, :], solUP[i][2, :],
            color='#f87171', lw=lw3d, alpha=0.5, zorder=1)
    ax.plot(solUM[i][0, :], solUM[i][1, :], solUM[i][2, :],
            color='#ef4444', lw=lw3d, alpha=0.5, zorder=1)

# --- legend with proxy artists ---
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='#ffffff', lw=2, label=f'L{Lset}
        {family.capitalize()} Orbit'),
    Line2D([0], [0], color='#4ade80', lw=1.5, label='Stable + branch'),
    Line2D([0], [0], color='#22c55e', lw=1.5, label='Stable - branch'),
    Line2D([0], [0], color='#f87171', lw=1.5, label='Unstable + branch'),
    Line2D([0], [0], color='#ef4444', lw=1.5, label='Unstable - branch'),
    Line2D([0], [0], marker='o', color='w', markerfacecolor='#2563eb',
            markersize=8, label='Earth', linestyle='None'),
    Line2D([0], [0], marker='o', color='w', markerfacecolor='#d1d5db',
            markersize=6, label='Moon', linestyle='None'),
]

```

```

ax.legend(handles=legend_elements,
          facecolor='#1e293b', edgecolor='#334155',
          labelcolor='#cbd5e1', fontsize=9,
          loc='upper left', bbox_to_anchor=(1.02, 1))

# --- styling ---
ax.set_xlabel('x [nd]', color='#94a3b8', fontsize=11, labelpad=8)
ax.set_ylabel('y [nd]', color='#94a3b8', fontsize=11, labelpad=8)
ax.set_zlabel('z [nd]', color='#94a3b8', fontsize=11, labelpad=8)
ax.set_title(f'L{Lset} {family.capitalize()} - Invariant Manifolds 3D ( =
            {info_act["stability"]:.2f})',
            color='#e2e8f0', fontsize=13, pad=15)

# dark theme for 3D axes
ax.xaxis.pane.fill = False
ax.yaxis.pane.fill = False
ax.zaxis.pane.fill = False
ax.xaxis.pane.set_edgecolor('#1e293b')
ax.yaxis.pane.set_edgecolor('#1e293b')
ax.zaxis.pane.set_edgecolor('#1e293b')
ax.grid(True, color='#1e293b', lw=0.5)
ax.tick_params(colors='#64748b', labelsize=9)

# --- CAMERA POSITION
ax.view_init(elev=elev, azimuth=azim)
ax.set_box_aspect([1, 1, 0.5])

plt.tight_layout()
fig.subplots_adjust(right=0.80)
plt.savefig('manifolds_3d_static.png', dpi=300, bbox_inches='tight',
          facecolor='#0d1117')

```

```

print("Saved to manifolds_3d_static.png")
plt.show()

##### 2D PLOT - ORBIT + MANIFOLDS #####

fig, ax = plt.subplots(figsize=(10, 8), facecolor='#0d1117')
ax.set_facecolor('#0d1117')

# --- periodic orbit ---
'''
orbit_sol = propagate_3bp(S0_act, 0.0, period_act, npts_fine)
ax.plot(orbit_sol.y[0], orbit_sol.y[1],
        color='#ffffff', lw=2, zorder=10, label='L1 Halo Orbit')
'''

# --- Earth & Moon ---
earth = plt.Circle((-mu, 0), 0.025, color='#2563eb', zorder=15)
ax.add_patch(earth)
ax.text(-mu, -0.06, 'Earth', color='#93c5fd', ha='center', fontsize=9)

moon = plt.Circle((1 - mu, 0), 0.012, color='#d1d5db', zorder=15)
ax.add_patch(moon)
ax.text(1 - mu, -0.045, 'Moon', color='#d1d5db', ha='center', fontsize=9)

# --- L1 / L2 points ---
for lx, lname in [(0.8369, 'L1'), (1.1557, 'L2')]:
    ax.plot(lx, 0, '+', color='#a78bfa', ms=8, mew=1.4, zorder=12)
    ax.text(lx, 0.04, lname, color='#a78bfa', ha='center', fontsize=8)

```

```

# --- manifolds - plot all trajectories from each seeded point ---
# use thin lines + alpha for clarity when overlapping

for i in range(npts_period):
    # stable manifolds - backward in time, approach the orbit
    # MODIFIED: List indexing for 2D plotting
    ax.plot(solSP[i][0, :], solSP[i][1, :],
            color='#4ade80', lw=0.3, alpha=0.6, zorder=2)
    ax.plot(solSM[i][0, :], solSM[i][1, :],
            color='#22c55e', lw=0.3, alpha=0.6, zorder=2)

    # unstable manifolds - forward in time, depart the orbit
    ax.plot(solUP[i][0, :], solUP[i][1, :],
            color='#f87171', lw=0.3, alpha=0.6, zorder=1)
    ax.plot(solUM[i][0, :], solUM[i][1, :],
            color='#ef4444', lw=0.3, alpha=0.6, zorder=1)

# --- legend with proxy artists ---
from matplotlib.lines import Line2D
legend_elements = [
    Line2D([0], [0], color='#ffffff', lw=2, label='L1 Halo Orbit'),
    Line2D([0], [0], color='#4ade80', lw=1.5, label='Stable + branch'),
    Line2D([0], [0], color='#22c55e', lw=1.5, label='Stable - branch'),
    Line2D([0], [0], color='#f87171', lw=1.5, label='Unstable + branch'),
    Line2D([0], [0], color='#ef4444', lw=1.5, label='Unstable - branch'),
    plt.Circle((0, 0), 0.02, color='#2563eb', label='Earth'),
    plt.Circle((0, 0), 0.01, color='#d1d5db', label='Moon'),
]

ax.legend(handles=legend_elements,
          facecolor='#1e293b', edgecolor='#334155',

```

```

        labelcolor='#cbd5e1', fontsize=9,
        loc='upper left', bbox_to_anchor=(1.02, 1))

# --- axes / styling ---
ax.set_xlabel('x [nd]', color='#94a3b8', fontsize=11)
ax.set_ylabel('y [nd]', color='#94a3b8', fontsize=11)
ax.set_title(f'L{Lset} {family} - Invariant Manifolds ( =
    {info_act["stability"]:.2f})',
             color='#e2e8f0', fontsize=13, pad=12)
ax.tick_params(colors='#64748b')
for spine in ax.spines.values():
    spine.set_edgecolor('#1e293b')
ax.set_aspect('equal')
ax.grid(True, color='#1e293b', lw=0.6, zorder=0)

plt.tight_layout()
fig.subplots_adjust(right=0.75)
plt.tight_layout(rect=[0, 0, 0.75, 1])
plt.savefig('invariant_manifolds.png', dpi=600, bbox_inches='tight',
           facecolor='#0d1117')
plt.show()

```